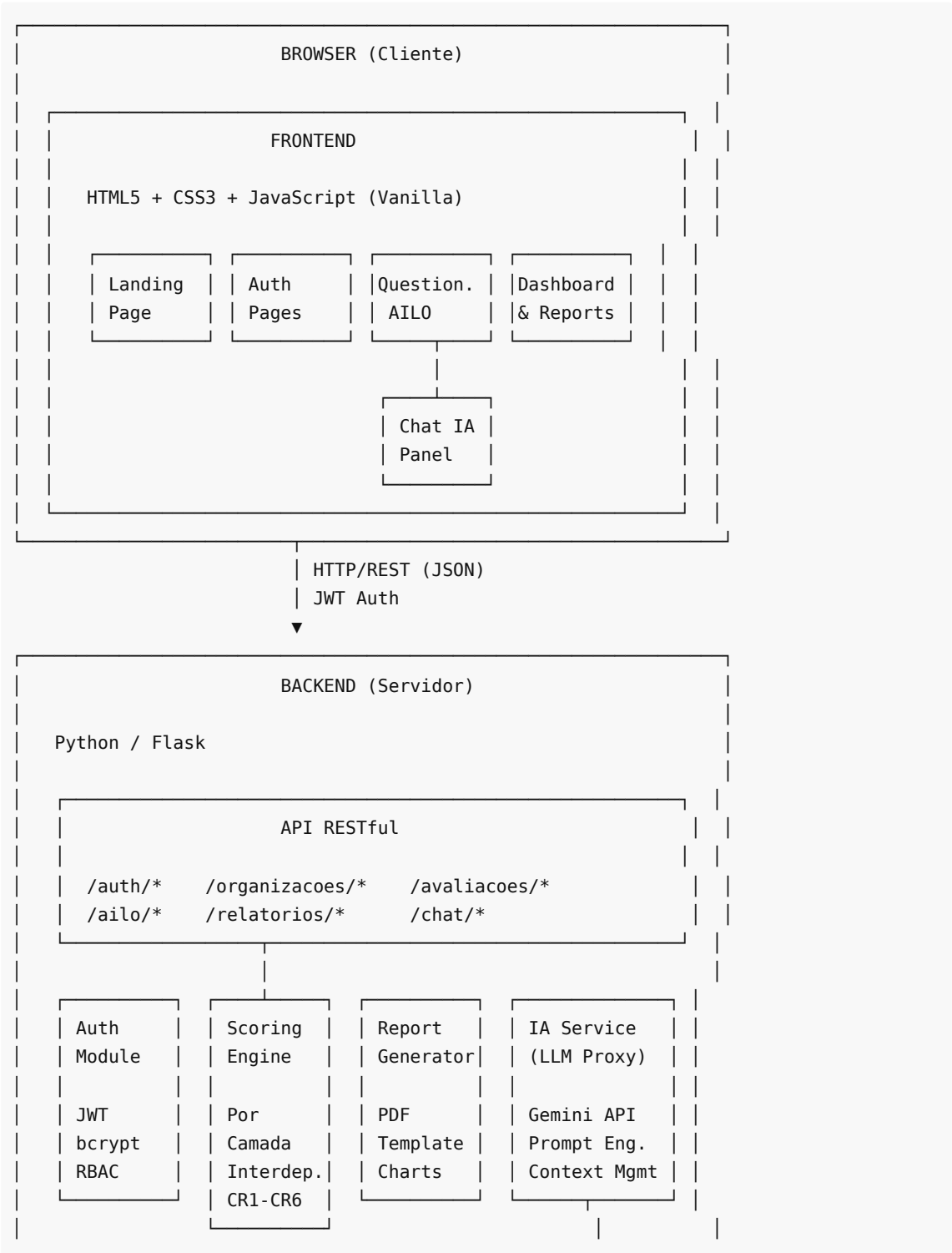
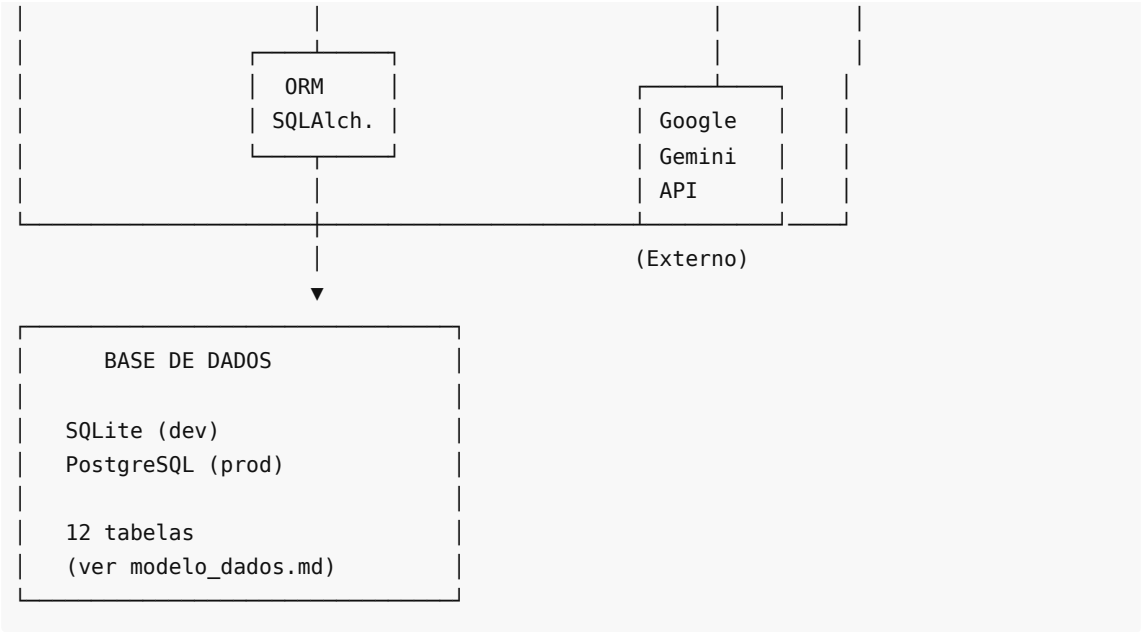


Arquitetura do Sistema

1. Visão Geral

A arquitetura segue um padrão de **3 camadas** (frontend, backend, base de dados) com integração externa a um serviço LLM. A escolha reflete as necessidades acadêmicas do projeto — simplicidade de deployment, facilidade de manutenção e alinhamento com as disciplinas do curso.





2. Stack Tecnológica

Camada	Tecnologia	Justificação
Frontend	HTML5, CSS3, JavaScript (Vanilla)	Simplicidade, sem dependências de build. Compatível com disciplinas do curso (Lab. Sistemas Web).
Backend	Python 3.11+ / Flask	Leve, flexível, ecossistema Python ideal para integração com IA.
ORM	SQLAlchemy	Abstração da BD, migrations, seed data.
BD (dev)	SQLite	Zero configuração, ficheiro único, ideal para desenvolvimento e demonstração.
BD (prod)	PostgreSQL	Robusto, escalável, suporta JSON nativo e concorrência.
Auth	JWT (PyJWT) + bcrypt	Standard da indústria para APIs RESTful.
LLM	Google Gemini API	Custo acessível, boa performance em português, API simples.
PDF	ReportLab ou WeasyPrint	Geração de PDFs profissionais a partir de templates.
Charts	Chart.js (frontend)	Leve, responsivo, ideal para gráficos radar/hexagonais.
Versionamento	Git + GitHub	Controlo de versões e entrega ao professor.

3. Estrutura de Pastas

```
Projeto_iAL0/  
├── backend/
```

```

├── app/
│   ├── __init__.py          # Flask app factory
│   ├── config.py            # Configurações (dev/prod)
│   ├── models/              # Modelos SQLAlchemy
│   │   ├── __init__.py
│   │   ├── utilizador.py
│   │   ├── organizacao.py
│   │   ├── ailo.py          # Camadas, Componentes, Indicadores
│   │   ├── avaliacao.py
│   │   ├── resposta.py
│   │   ├── resultado.py
│   │   ├── ferramenta.py
│   │   └── conversa.py
│   ├── routes/              # Endpoints da API
│   │   ├── __init__.py
│   │   ├── auth.py
│   │   ├── organizacoes.py
│   │   ├── ailo.py
│   │   ├── avaliacoes.py
│   │   ├── respostas.py
│   │   ├── resultados.py
│   │   ├── chat.py
│   │   └── relatorios.py
│   ├── services/            # Lógica de negócio
│   │   ├── __init__.py
│   │   ├── scoring.py       # Motor de scoring AILO
│   │   ├── interdependencias.py
│   │   ├── ia_assistant.py  # Integração Gemini
│   │   ├── report_generator.py
│   │   └── recomendacoes.py
│   └── utils/
│       ├── auth.py          # JWT helpers
│       ├── decorators.py    # @login_required, @admin_required
│       └── validators.py
├── migrations/              # Alembic migrations
├── seeds/
│   ├── camadas.py           # Seed das 6 camadas
│   ├── componentes.py       # Seed dos 23 componentes
│   ├── indicadores.py        # Seed dos 51 indicadores
│   └── ferramentas.py       # Seed do catálogo de ferramentas
├── tests/
│   ├── test_auth.py
│   ├── test_scoring.py
│   ├── test_api.py
│   └── test_interdependencias.py
├── .env.example
├── requirements.txt
├── run.py
└── seed.py                  # Script para popular BD

├── frontend/
└── index.html               # Landing page

```

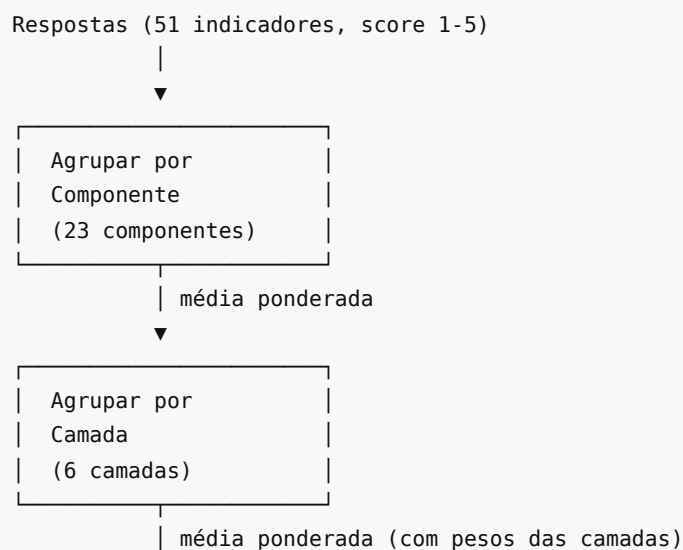
```

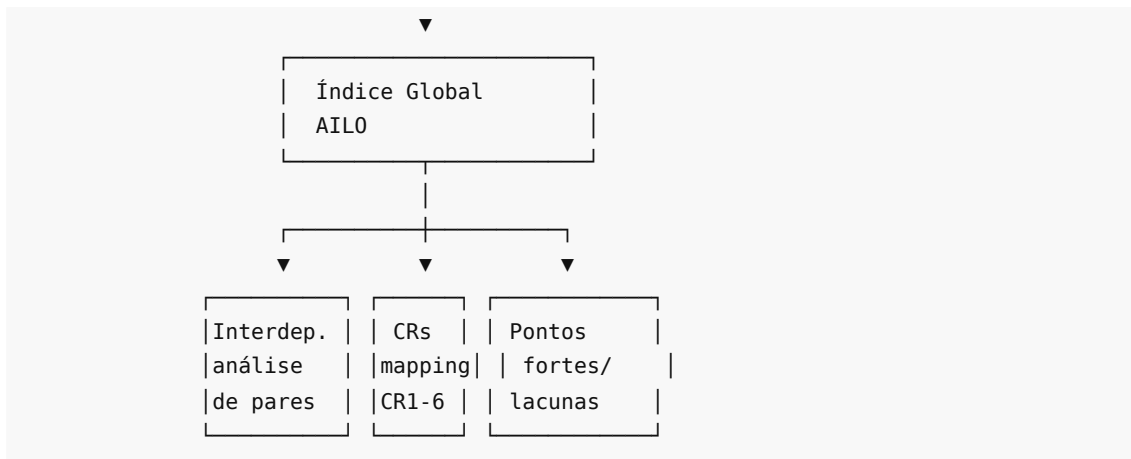
|   ├── css/
|   |   ├── main.css          # Design system
|   |   ├── questionario.css
|   |   ├── dashboard.css
|   |   └── relatorio.css
|   ├── js/
|   |   ├── app.js            # Routing e inicialização
|   |   ├── api.js            # Wrapper para chamadas à API
|   |   ├── auth.js           # Login/registo
|   |   ├── questionario.js    # Lógica do questionário por camada
|   |   ├── chat.js           # Chat com assistente IA
|   |   ├── dashboard.js       # Gráficos e visualizações
|   |   └── relatorio.js        # Visualização de relatório
|   └── pages/
|       ├── login.html
|       ├── register.html
|       ├── organizacoes.html
|       ├── questionario.html
|       ├── dashboard.html
|       └── relatorio.html
|
|   ├── docs/                  # Documentação do projeto
|   |   └── Fase_1/            # (esta pasta)
|   |
|   ├── Fase_1/                # Deliverables da Fase 1
|   ├── IALO_Eng.docx          # Framework do professor
|   ├── IALO_pt.docx
|   ├── Relatorio_Desenvolvimento.docx
|   ├── relatório_inicial_IALO.docx
|   └── .gitignore

```

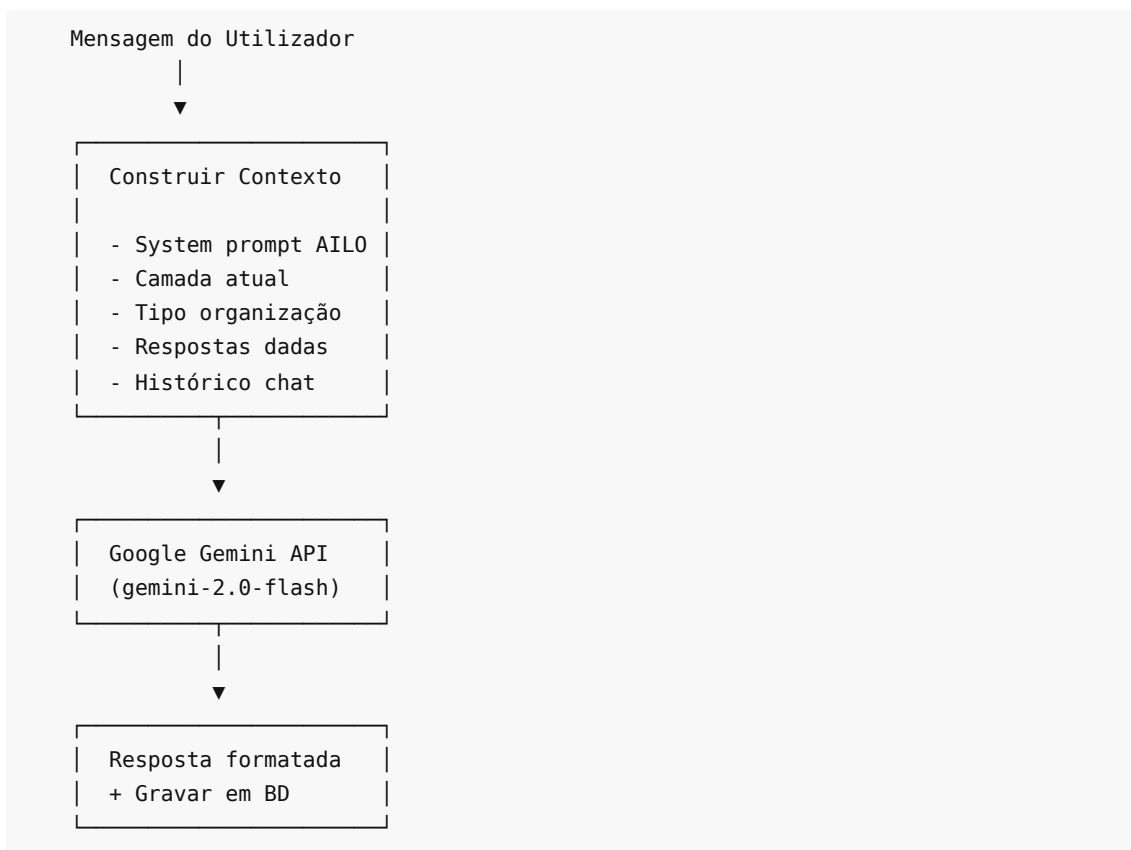
4. Módulos Principais

4.1. Motor de Scoring (services/scoring.py)

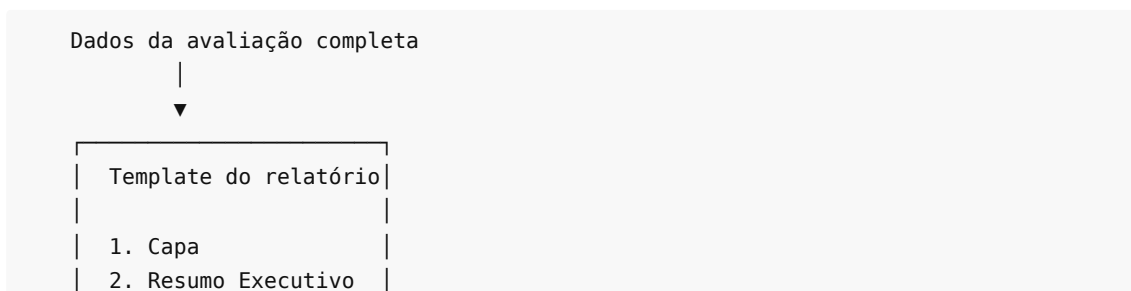


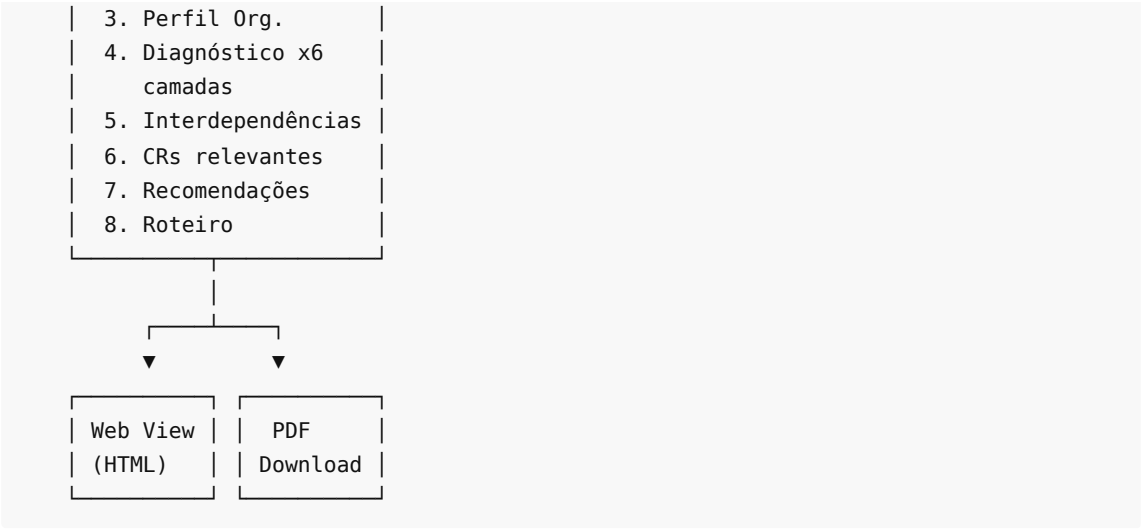


4.2. Assistente IA (`services/ia_assistant.py`)



4.3. Gerador de Relatórios (`services/report_generator.py`)





5. Padrões e Decisões de Design

Decisão	Justificação
API RESTful	Standard, testável, desacoplado do frontend
JWT stateless	Sem sessões no servidor, escalável
SQLAlchemy ORM	Abstração da BD, código Python limpo, migrations
Service layer	Separação entre routes (HTTP) e lógica de negócio
Seed data	Framework AILO carregado na BD na primeira execução
Vanilla JS	Sem frameworks frontend — simplicidade e alinhamento com disciplinas
Chart.js	Única dependência frontend — gráficos radar para o hexágono AILO
Gemini API	Custo acessível, boa qualidade em PT, API simples

6. Segurança

Mecanismo	Implementação
Autenticação	JWT com expiração de 24h. Refresh via re-login.
Password hashing	bcrypt com salt rounds ≥ 12
Autorização	Decorator <code>@login_required</code> em todas as rotas protegidas
RBAC	Papel admin / utilizador verificado por decorator
Input validation	Validação no backend antes de queries à BD
SQL Injection	Prevenido pelo ORM (parametrized queries)
XSS	Escape de HTML em outputs. Content-Security-Policy headers.

CORS	Configurado para permitir apenas origens autorizadas
API keys	Chave Gemini apenas no backend (.env), nunca no frontend

7. Deployment (Demonstração)



Para demonstração académica, o Flask serve também os ficheiros estáticos do frontend. Em produção, seria separado com Nginx.